Московский государственный технический

университет им. Н.Э. Баумана

Факультет «Информатика и системы управления»

Кафедра ИУ5 «Системы обработки информации и управления»

Курс «Парадигмы и конструкции языков программирования»

Отчет по рубежному контролю №2

«Модульное тестирование»

Выполнил:

Студент группы ИУ5-31Б

Ростинин Марк

Проверил:

Гапанюк Ю.Е.

2025 г.

Листинг кода:

Файл main.py:

```
from dataclasses import dataclass, field
from typing import Optional, List, Dict, Tuple
import random


@dataclass
class File:
    file_id: int
    file_size: int
    folder_id: Optional[int] = None


@dataclass
class Folder:
    folder_id: int
    name: str
    files: List[File] = field(default_factory=list)


@dataclass
class FolderFileLink:
    folder_id: int
    file_id: int


def get_sorted_folders_with_files(folders: List[Folder]) -> List[Folder]:
    return sorted(folders, key=lambda f: f.name)


def get_folders_total_size(folders: List[Folder]) -> List[Tuple[str, int]]:
    totals = [(folder.name, sum(f.file_size for f in folder.files)) for
folder in folders]
    return sorted(totals, key=lambda x: x[1], reverse=True)


def get_linked_files_for_even_folders(folders: List[Folder], files: Dict[int,
File], links: List[FolderFileLink]) -> \
Dict[str, List[File]]:
    res = {}
    even_folders = [
        f for f in folders
        if any(ch.isdigit() and int(ch) % 2 == 0 for ch in f.name)
    ]
    for folder in even_folders:
        linked_file_ids = [link.file_id for link in links if link.folder_id
== folder.folder_id]
        res[folder.name] = [files[fid] for fid in linked_file_ids]
    return res


def generate_data(num_folders: int = 5, files_per_folder: int = 5):
    folders: List[Folder] = []
    files: Dict[int, File] = {}
    links: List[FolderFileLink] = []
    file_counter = 1
    for i in range(1, num_folders + 1):
        folder = Folder(folder_id=i, name=f"Folder_{i}")
        for j in range(1, files_per_folder + 1):
            f = File(file_id=file_counter, file_size=random.randint(100,
```

```
10_000), folder_id=i)
            folder.files.append(f)
            files[file_counter] = f
            file_counter += 1
        folders.append(folder)
    for _ in range(10):
        folder = random.choice(folders)
        file = random.choice(list(files.values()))
        link = FolderFileLink(folder_id=folder.folder_id,
file_id=file.file_id)
        if link not in links: links.append(link)
    return folders, files, links


if __name__ == "__main__":
    f, fls, l = generate_data()
    print("Данные успешно сгенерированы и обработаны.")
```

Файл test_main.py:

```
import pytest
from main import File, Folder, FolderFileLink, \
    get_sorted_folders_with_files, \
    get_folders_total_size, \
    get_linked_files_for_even_folders


def test_sorting_folders_by_name():
    f1 = Folder(1, "Gamma")
    f2 = Folder(2, "Alpha")
    f3 = Folder(3, "Beta")

    result = get_sorted_folders_with_files([f1, f2, f3])

    assert result[0].name == "Alpha"
    assert result[1].name == "Beta"
    assert result[2].name == "Gamma"


def test_total_size_calculation():
    folder1 = Folder(1, "Small", files=[File(1, 100)])
    folder2 = Folder(2, "Big", files=[File(2, 200), File(3, 300)])

    result = get_folders_total_size([folder1, folder2])

    assert result[0] == ("Big", 500)
    assert result[1] == ("Small", 100)


def test_even_folders_links():
    f_even = Folder(2, "Folder_2")
    f_odd = Folder(1, "Folder_1")

    file_linked = File(10, 500)
    files_map = {10: file_linked}

    links = [FolderFileLink(folder_id=2, file_id=10)]

    result = get_linked_files_for_even_folders([f_even, f_odd], files_map,
links)

    assert "Folder_2" in result
```

```
    assert "Folder_1" not in result
    assert result["Folder_2"][0].file_id == 10
```